

Table des matières

Synthèse (avancée) — « Aurore », une boutique en ligne, de bout en bout	1
Étape 0 — Le projet et son architecture (<i>démarche du Cours 00</i>)	2
Étape 1 — Charger le brut dans RAW (deux sources) (<i>démarche du Cours 01</i>).....	4
Étape 2 — Concevoir la constellation (<i>démarche du Cours 02</i>).....	8
Étape 3 — La couche STAGING (<i>démarche du Cours 03</i>)	9
Étape 4 — INTERMEDIATE, MARTS, tests et doc (<i>démarche du Cours 04</i>).....	11
Avant-goût — la couche PRESENTATION (<i>approfondie au Cours 06</i>)	17
Récapitulatif — ce que la synthèse a rejoué, et ce qu'elle a montré en plus.....	18

Synthèse (avancée) — « Aurore », une boutique en ligne, de bout en bout

Projet synthèse avancé — Formation Analytics Engineering (démarches des Cours 00 → 04)

Support d'avant la refonte. La formation a été refondue le 2026-09-20, de Snowflake vers DuckDB et dbt-duckdb, sur des univers redessinés. Cette synthèse porte encore l'ancienne pile, et son projet rejouable a été retiré du dossier en attendant sa réécriture. Les cours 01 à 06, eux, sont à jour. Voir `README.md`.

Ce que c'est, et comment le lire. Les cours vous ont fait bâtir Boréal (vos labos). Deux synthèses referment ensuite la formation : La Brûlerie (facile) d'abord (le pipeline dépouillé, une caisse, chargée par dbt `seed`), puis Aurore, celle-ci, la plus riche. Ce document résout un projet de bout en bout (une boutique en ligne) dont le seul but est de refaire tout le trajet (du brut à l'étoile testée) sur un cas délibérément plus riche. On ne réexplique pas la théorie : on exécute les démarches des Cours 00 à 04, en pointant au passage la section de cours qui les fonde. Gardez l'aide-mémoire ouvert à côté.

L'intérêt d'un cas plus complexe. Boréal et La Brûlerie n'ont qu'un seul processus métier (la vente), donc une seule étoile et une seule couche de marts. Boréal pose bien une couche `INTERMEDIATE` (l'enrichissement coût/marge), mais, avec un seul fait, il n'en montre qu'un usage minimal. Le vrai monde a plusieurs processus, et souvent plusieurs sources. Aurore suit quatre processus (vendre, expédier, rembourser et consulter), ce qui fait apparaître, *pour de vrai*, l'usage riche de deux notions que les cours n'ont fait qu'effleurer : la couche `INTERMEDIATE` réutilisée par plusieurs faits (entre le staging et les marts) et plusieurs schémas de marts, découpés par domaine (les *data marts*), reliés par des dimensions conformes. On finit par un avant-goût de la couche `PRESENTATION`.

Et deux systèmes sources. Aurore ne tire pas tout de la base de son appli : un outil d'analyse web (un SaaS distinct) exporte les pages vues du catalogue. Deux sources → deux schémas de RAW et de STAGING, un par source (le cas `raw_erp/raw_web` du Cours 00, ici RAW et RAW_WEB), reliés par des dimensions conformes inter-sources, une même `dim_produit` sert un fait venu de l'appli et un fait venu du web.

Où l'on s'arrête. Ce document couvre les Cours 00 → 04 (jusqu'aux marts testés et documentés). L'ELT/orchestration (05), Power BI (06) et la CI/CD (07) ne sont pas traités ici.

Trois projets, toujours jamais mêlés. Boréal (vos labos, source SAP), et les deux synthèses de clôture, La Brûlerie (facile, une caisse) et Aurore (avancée, une boutique en ligne). Même métier de fond (*une entreprise vend des produits à des clients*), trois domaines distincts, et trois voies d’entrée distinctes : SAP par COPY INTO (Boréal), seeds figés (La Brûlerie), générateur multi-source (Aurore).

Étape 0 — Le projet et son architecture (démarche du Cours 00)

0.1 L’entreprise : « Aurore »

Aurore est une boutique en ligne (fictive) qui vend des articles de maison et d’électronique au grand public partout au Canada. Sa source, ce n’est pas un ERP SAP : c’est la base de son application web, une base OLTP normalisée (revoir *Théorie 02, section 1*), exportée en CSV et chargée telle quelle. Contrairement à Boréal (un seul acte : vendre au concessionnaire), Aurore enchaîne plusieurs activités (et les tire de deux systèmes sources) qu’on veut analyser séparément :

Processus métier	L’événement qu’on mesure	Question d’analyste typique
Vendre	une ligne de commande passée sur le site	« ventes nettes par catégorie et par mois ? »
Expédier	une ligne expédiée d’un entrepôt	« délai de traitement et volume par entrepôt ? »
Rembourser	une ligne retournée	« taux de retour par produit et par motif ? »
Consulter (2 ^e source)	une page produit vue sur le site	« quels produits sont les plus consultés, et se vendent-ils ? »

Ces activités viennent de deux systèmes sources distincts : les trois premières de la base de l’appli (OLTP), la dernière d’un outil d’analyse web (un SaaS qui exporte les pages vues). Deux sources, c’est ce qui fera apparaître plusieurs schémas de RAW et de STAGING, un par source (section 0.2). C’est toute la différence avec les projets des cours, et la raison d’être de ce document.

0.2 L’architecture cible — la version riche

Le squelette reste celui du Cours 00 : RAW → STAGING → MARTS → BI. Mais un projet à plusieurs processus insère et dédouble des couches. Voici le plan complet d’Aurore, chaque schéma Snowflake qu’on va bâtir, et à quelle étape :

Couche (type de schéma)	Schéma(s) Snowflake	Rôle	Convention	Bâtie à l’étape
RAW (<i>par source</i>)	RAW (appli), RAW_WEB (analytique)	le brut, tel que chaque source l’a exporté	(<i>noms de la source</i>)	1 (Cours 01)
STAGING (<i>par source</i>)	STAGING (appli), STAGING_WEB	nettoyage 1:1 (renommer, typer, décoder)	stg_<table> / stg_web_<table>	3 (Cours 03)

Couche (type de schéma)	Schéma(s) Snowflake	Rôle	Convention	Bâtie à l'étape
INTERMEDIATE	INTERMEDIATE	logique métier réutilisable entre staging et marts	int_< sujet >	4 (Cours 04)
MARTS (<i>par domaine</i>)	MARTS_VENTES, MARTS_LOGISTIQUE, MARTS_MARKETING	les étoiles métier, une par domaine	dim_* / fct_*	4 (Cours 04)
PRESENTATION	PRESENTATION	agrégats taillés pour un rapport précis	rpt_< rapport >	avant-goût (Cours 06)
(<i>historique</i>)	SNAPSHOTS	versions SCD2 d'une dimension	(<i>snapshot dbt</i>)	4 (Cours 04)

Le point à retenir de toute cette synthèse. Les trois couches du Cours 00 (RAW, STAGING, MARTS) sont la colonne vertébrale, jamais négociable. Mais ce ne sont ni les seuls types de couches, ni des schémas uniques :

- D'autres types existent. Entre le staging et les marts, on peut intercaler une couche INTERMEDIATE (int_*) pour la logique réutilisable ; au-dessus des marts, une couche PRESENTATION (rpt_*) pour des tables prêtes à afficher. Optionnelles : on les ajoute quand la complexité le justifie, Boréal pose un int_ ; Aurore les multiplie (int_ réutilisé par plusieurs faits, plusieurs marts, et une PRESENTATION).
- Un même type peut se dédoubler — de deux façons. *Par source* : plusieurs RAW et STAGING, un par système source (Aurore en a deux : l'appli → RAW/STAGING, l'analytique web → RAW_WEB/STAGING_WEB). *Par domaine* : plusieurs MARTS (MARTS_VENTES, MARTS_LOGISTIQUE, MARTS_MARKETING). Ce qui relie ces schémas sans les faire diverger : les dimensions conformes (une seule dim_produit, une seule dim_date), partagées non seulement entre domaines, mais entre sources (le fait web et le fait ventes pointent la même dim_produit).

Autrement dit : RAW → STAGING → MARTS est la règle ; INTERMEDIATE, PRESENTATION et le découpage des marts par domaine sont des outils qu'on sort au besoin. Aurore les sort tous.

Le lignage d'Aurore, en une ligne : {RAW, RAW_WEB} → {STAGING, STAGING_WEB} → INTERMEDIATE → {MARTS_VENTES, MARTS_LOGISTIQUE, MARTS_MARKETING} → PRESENTATION → Power BI, le tout versionné (Git) et testé (dbt).

0.3 Le projet dbt

Comme Boréal (boreal) et La Brûlerie (brulerie), Aurore vit dans son propre projet dbt, aurore, sur la base AUREORE, créé par dbt init aurore, exactement comme au Cours 00. Même pile, même profiles.yml en env_var, même env conda formations. On ne remélange jamais les trois.

Son dbt_project.yml a le même en-tête que les deux autres (identité, chemins, écrits par dbt init) et, comme Boréal, aucun bloc seeds : le brut d'Aurore entre par COPY INTO, donc il se déclare en sources. Ce qui le distingue tient au bloc models : , sensiblement plus fourni, sept dossiers de modèles, là où Boréal en a trois et La Brûlerie deux (Étape 4.B).

Étape 1 — Charger le brut dans RAW (deux sources) (démarche du Cours 01)

Chaque projet de la formation a sa propre voie d'entrée : Boréal charge ses extraits SAP par `COPY INTO`, La Brûlerie fige son brut en seeds. Aurore, lui, a **deux** systèmes sources, il lui faut donc son propre générateur multi-source, `generer_aurore.py` : il fabrique les deux jeux d'extraits dans le style « moche » d'un vrai export (noms techniques, dates en texte, codes à décoder), un par source. C'est l'équivalent à deux sources du `generer_sources.py` de Boréal.

Aurore a deux sources. La principale (la base de l'appli) a exporté 13 tables en CSV, chargées telles quelles dans RAW (stage + `COPY INTO`, tout en `STRING`, exactement comme au Cours 01). C'est une base OLTP normalisée : chaque fait est écrit une fois, les tables se pointent par des `..._id`. La seconde source (l'analytique web) a son propre schéma de brut, `RAW_WEB` (section 1.2 ci-dessous).

L'inventaire de la source « appli » (RAW) (le grain d'origine entre parenthèses) :

Table RAW	Contenu	Sert surtout à...
client (1 client)	cli_id, courriel, prov, date_insc texte, seg (B2C/B2B)	dim_client
adresse (1 adresse)	adr_id, cli_id, type (F/L), ville, prov, code_postal	dim_client (adresse de facturation)
categorie (1 catégorie)	cat_id, cat_nom, cat_parent (auto-référence → hiérarchie)	dim_produit
fournisseur (1 fourn.)	four_id, four_nom, pays	dim_produit
produit (1 produit)	prod_id, prod_nom, cat_id, four_id, prix_liste texte, actif (O/N)	dim_produit
entrepot (1 entrepôt)	ent_id, ent_nom, prov	dim_entrepot
promotion (1 code)	promo_code, type (PCT/FIXE), valeur texte, actif	dim_promotion
commande (1 en-tête)	cmd_id, cli_id, cmd_ts texte, statut (code), code_promo (nullable)	ventes
commande_ligne (1 ligne)	cmd_id, lig_no, prod_id, qte, prix_unit, remise_lig (texte)	grain des ventes

Table RAW	Contenu	Sert surtout à...
paiement (1 paiement)	pai_id, cmd_id, mode, montant texte, statut (OK/REFU)	contrôle/finance
expedition (1 en-tête d'envoi)	exp_id, cmd_id, ent_id, exp_ts texte, transporteur, statut	expéditions
expedition_ligne (1 ligne d'envoi)	exp_id, lig_no, qte_exp texte	grain des expéditions
retour (1 ligne de retour)	ret_id, cmd_id, lig_no, qte_ret, motif, ret_ts, montant_rembourse (texte)	grain des retours

Pour suivre les chiffres du doigt dans tout le document, on garde un tout petit échantillon, deux commandes. Voici les tables clés (les autres suivent le même moule) :

RAW.client

cli_id	prov	seg
42	QC	B2C
57	ON	B2B

RAW.produit (*prix en texte, cat_id pointe vers categorie*)

prod_id	prod_nom	cat_id	four_id	prix_liste
5	Bouilloire	11 (Cuisine)	1	"40.00"
8	Casque audio	20 (Électronique)	2	"100.00"
9	Tasse	11 (Cuisine)	1	"12.00"

RAW.categorie (*hiérarchie : cat_parent pointe vers une autre catégorie*)

cat_id	cat_nom	cat_parent
10	Maison	(rien)
11	Cuisine	10
20	Électronique	(rien)

RAW.commande (horodatage en texte, statut/code_promo codés)

cmd_id	cli_id	cmd_ts	statut	code_promo
100	42	"2024-05-10 09:30"	EX	PROMO10
101	57	"2024-05-12 14:00"	EX	(rien)

RAW.commande_ligne (le futur grain des ventes ; nombres en texte)

cmd_id	lig_no	prod_id	qte	prix_unit	remise_lig
100	1	5 (Bouilloire)	"2"	"40.00"	"0"
100	2	9 (Tasse)	"4"	"12.00"	"0"
101	1	8 (Casque)	"1"	"100.00"	"0.05"

RAW.promotion

promo_code	type	valeur	actif
PROMO10	PCT	"10"	O

RAW.paielement (une commande peut avoir plusieurs paiements)

pai_id	cmd_id	mode	montant	statut
900	100	CARTE	"100.00"	OK
901	100	CARTE_CADEAU	"15.20"	OK
902	101	CARTE	"95.00"	OK

RAW.expedition (la commande 100 part en deux envois, de deux entrepôts)

exp_id	cmd_id	ent_id	exp_ts	statut
800	100	1 (Montréal)	"2024-05-11"	LIV
801	100	2 (Toronto)	"2024-05-13"	LIV
802	101	2 (Toronto)	"2024-05-13"	EXP

RAW.expedition_ligne (Lig_no pointe la ligne de commande_ligne ; le produit n’y est pas)

exp_id	lig_no	qte_exp
800	1	"2"
800	2	"2"
801	2	"2"
802	1	"1"

La commande 100 a 4 tasses (ligne 2) : 2 partent de Montréal (envoi 800), 2 de Toronto (envoi 801). Voilà une expédition partielle, impossible à voir au grain « commande ».

RAW.retour (*Alice retourne 1 bouilloire défectueuse*)

ret_id	cmd_id	lig_no	qte_ret	motif	ret_ts	montant_rembourse
700	100	1	"1"	DEF	"2024-05-20"	"36.00"

Le fil rouge des chiffres. Commande 100 (Alice) : 128,00 \$ brut, moins 10 % (PROMO10) = 115,20 \$ net, d'où les paiements 100,00 + 15,20. Elle est expédiée en deux fois, puis 1 bouilloire est retournée, remboursée 36,00 \$ (le prix *net* unitaire, on verra pourquoi ce n'est pas 40,00). On va retrouver ces montants à chaque étape.

1.2 La seconde source : l'analytique web (RAW_WEB)

L'outil d'analyse web d'Aurore est un système à part (un SaaS externe, pas la base de l'appli). Il exporte un flux d'événements : chaque page produit vue sur le site. On le charge dans son propre schéma de brut, RAW_WEB, un schéma de RAW par source, exactement le cas raw_erp/raw_web du Cours 00.

RAW_WEB.page_vue (*le futur grain des consultations ; le produit est repéré par son SKU, qui vaut ici le prod_id de l'appli*)

pv_id	pv_ts	sku	session_id	duree_sec
5001	"2024-05-09 20:14"	5 (Bouilloire)	S-1	"42"
5002	"2024-05-10 09:12"	8 (Casque)	S-2	"88"
5003	"2024-05-10 09:20"	5 (Bouilloire)	S-2	"15"

Le pont entre deux sources : une clé métier commune. Le web désigne un produit par son sku ; l'appli, par son prod_id. Ici, Aurore utilise le prod_id comme SKU sur son site, donc les deux coïncident, la même clé métier de part et d'autre. C'est ça qui rend dim_produit conforme entre les deux sources : le fait web hachera produit_id et retombera exactement sur la clé de la dimension bâtie depuis l'appli. (*Si les clés différaient (un SKU maison ≠ prod_id), il faudrait une petite table de correspondance ; le principe conforme, lui, ne change pas.*)

Étape 2 — Concevoir la constellation (*démarche du Cours 02*)

Un seul processus donnait une étoile (Cours 02). Aurore en a quatre (dont un venu d'une autre source), qui partagent des dimensions : c'est une constellation (*faits reliés par des dimensions conformes*, revoir *Théorie 02, section 9*). On applique les 4 décisions de Kimball à chaque processus.

2.1 Les grains (décisions 1-2, ×4)

Processus	Grain retenu (le plus fin)	Une ligne de fait =
Ventes	la ligne de commande	un produit commandé sur une commande
Expéditions	la ligne d'expédition	un produit expédié, d'un entrepôt, une date
Retours	la ligne de retour	un produit retourné sur une commande
Consultations (<i>source web</i>)	la page vue	un produit consulté, une session, un instant

Quatre grains différents, donc quatre faits séparés, jamais fusionnés. (Fusionner ventes et expéditions forcerait un grain bâtard : une même ligne vendue peut partir en deux envois, comme la tasse d'Alice. Les consultations, elles, viennent carrément d'une autre source.)

2.2 Les dimensions, et lesquelles sont conformes (décision 3)

On liste les axes d'analyse, puis on repère ceux partagés entre processus, ce seront les dimensions conformes (une seule table, réutilisée) :

Dimension	Ventes	Expéditions	Retours	Consultations	Conforme ?
dim_date	✓	✓	✓	✓	oui (les 4)
dim_produit	✓	✓	✓	✓	oui (les 4, 2 sources)
dim_client	✓		✓		oui (ventes + retours)
dim_entrepot		✓			non (logistique seule)
dim_promotion	✓				non (ventes seule)

C'est la bus matrix de Kimball (*Théorie 02, section 9*) : la grille processus × dimensions. Elle pilote le découpage en marts : les dimensions conformes vivront une seule fois et serviront plusieurs marts.

dim_produit conforme, en pratique — et même entre sources. Elle aplatit produit → categorie → fournisseur, en remontant la hiérarchie de catégories (cat_id → cat_parent : Cuisine → Maison). Cette même table sert les ventes, les expéditions, les retours et les consultations web, alors que ces dernières viennent d'une autre source. Résultat : « quels produits sont consultés mais peu vendus ? » se répond en croisant fct_pages_vues et fct_ventes sur dim_produit, sans réconciliation, c'est tout l'intérêt du conforme, poussé jusqu'à relier deux systèmes.

2.3 Les faits et leurs mesures (décision 4)

Fait	Mesures (au grain)	Additivité
fct_ventes	quantite, montant_brut, montant_remise, montant_net	additives ; taux_remise non additif
fct_expéditions	quantite_expediee, delai_traitement_jours	quantité additive ; le délai non (on le moyenne)
fct_retours	quantite_retournee, montant_rembourse	additives
fct_pages_vues (<i>source web</i>)	duree_sec (temps sur la page) ; une ligne = une vue	duree_sec sommable (total) ou moyennée

2.4 Le SCD (décision 3, suite)

Une seule dimension bouge assez pour justifier l'historique : dim_client, dont le segment (B2C → B2B) et la province changent parfois. On l'historisera en SCD2 (snapshot, Étape 4), comme le palier de Boréal ou le segment de Carla. Les autres dimensions seront traitées en SCD1 (on écrase), suffisant ici.

Le plan sur papier est complet. Quatre étoiles (dont une d'une 2^e source), cinq dimensions (dont dim_date/dim_produit conformes entre sources), un SCD2. On passe à dbt.

Étape 3 — La couche STAGING (*démarche du Cours 03*)

Rien de neuf par rapport au Cours 03 : une vue stg_* par table du brut, 1:1, qui renomme + type + decode, sans jointure. Avec 13 tables sources, on obtient 13 vues. On ne les écrit pas toutes ici, le moule est toujours le même (with source as (...) select ... rename). Deux suffisent à montrer le travail réel (typer, decoder) :

```
-- models/staging/stg_commande.sql → L'horodatage texte redevient une vraie date, Le statut se decode
with source as ( select * from {{ source('appli', 'commande') }} )
select
```

```

cmd_id          as commande_id,
cli_id          as client_id,
to_timestamp(cmd_ts, 'YYYY-MM-DD HH24:MI') as horodatage_commande,
to_date(to_timestamp(cmd_ts, 'YYYY-MM-DD HH24:MI')) as date_commande,
case statut when 'EX' then 'Expédiée'
            when 'AN' then 'Annulée'
            when 'PA' then 'En attente' end as statut,
nullif(code_promo, '') as code_promo -- '' → NULL (pas de promo)
from source

-- models/staging/stg_commande_ligne.sql → Les nombres texte redeviennent des nombres (Le futur grain)
with source as ( select * from {{ source('appli', 'commande_ligne') }} )
select
    cmd_id          as commande_id,
    lig_no          as ligne_num,
    prod_id         as produit_id,
    to_number(qte)   as quantite,
    to_decimal(prix_unit,10,2) as prix_unitaire,
    to_decimal(remise_lig,4,2) as taux_remise_ligne
from source

```

Les 11 autres suivent : stg_client (décode seg B2C/B2B), stg_produit (type prix_liste, décode actif O/N), stg_categorie, stg_fournisseur, stg_entrepot, stg_promotion (type valeur), stg_paiement, stg_expédition (type exp_ts), stg_expédition_ligne, stg_retour.

On respecte la règle « décodeur oui, joindre non » (Cours 03). stg_categorie garde cat_parent (le code du parent) sans aller le résoudre : remonter la hiérarchie Cuisine → Maison exige une jointure, ça se fera plus loin (dans l'intermediate/les marts). Le staging reste 1:1.

dbt run --select staging → 13 vues dans AURORA.STAGING. La façade propre de la source appli est posée.

La seconde source a son propre staging. Le flux web est déclaré comme une 2^e source dans _sources.yml (nom logique web, schéma RAW_WEB), et nettoyé par des modèles stg_web_* rangés dans leur propre schéma STAGING_WEB :

```

-- models/staging_web/stg_web_page_vue.sql → schéma STAGING_WEB
with source as ( select * from {{ source('web', 'page_vue') }} )
select
    pv_id          as page_vue_id,
    to_timestamp(pv_ts, 'YYYY-MM-DD HH24:MI') as horodatage_vue,
    to_date(to_timestamp(pv_ts, 'YYYY-MM-DD HH24:MI')) as date_vue,
    sku            as produit_id, -- Le SKU = Le prod_id de l'appli
    session_id,
    to_number(duree_sec) as duree_sec
from source

```

dbt run --select staging_web → 1 vue dans AURORE.STAGING_WEB. Deux sources, deux schémas de staging, chacun reste une façade 1:1 sur son brut. Remarquez sku as produit_id : on aligne le nom sur celui de l'appli dès le staging, pour que tout l'aval ne connaisse qu'une seule notion de « produit ».

Étape 4 — INTERMEDIATE, MARTS, tests et doc (démarche du Cours 04)

C'est ici que le cas riche se distingue vraiment. On procède en trois temps : (A) la couche intermediate, la nouveauté ; (B) les marts par domaine ; (C) les trois piliers (SCD2, tests, doc).

4.A La couche INTERMEDIATE — *pourquoi* elle apparaît ici

Le Cours 04 a introduit la couche INTERMEDIATE avec un int_unique chez Boréal (l'enrichissement coût/marge). Aurore, lui, a deux raisons classiques d'en intercaler plusieurs, des modèles int_* entre le staging et les marts :

1. De la logique réutilisée par plusieurs marts — on l'écrit une fois, pas deux.
2. Réduire une relation « plusieurs » (fan-in) avant d'atteindre le grain d'un fait.

Convention : int_< sujet >, rangés dans models/intermediate/, en ephemeral (dbt les injecte en CTE chez l'appelant, aucun objet dans l'entrepôt, ils ne servent qu'au pipeline).

La règle de décision. Un modèle mérite d'être intermediate s'il est (a) réutilisé par ≥ 2 modèles en aval, ou (b) une étape de préparation trop lourde pour un seul select de fait. Sinon, on laisse la logique dans le mart. On n'ajoute pas de couche pour le plaisir.

int_lignes_nettes — *la logique réutilisée (raison 1)*. Le montant net d'une ligne dépend d'une remise au niveau de la commande (le code promo), qu'il faut répartir sur les lignes. Cette logique sert fct_ventes (le net vendu) et fct_retours (on rembourse au prix *net*, pas au prix liste). Écrite deux fois, elle divergerait un jour. On l'isole :

```
-- models/intermediate/int_lignes_nettes.sql    (ephemeral)
with lignes as ( select * from {{ ref('stg_commande_ligne') }} ),
    entetes as ( select * from {{ ref('stg_commande') }} ),
    promos as ( select * from {{ ref('stg_promotion') }} ),

-- 1) brut par ligne, après la remise DE LIGNE éventuelle
base as (
    select l.commande_id, l.ligne_num, l.produit_id, l.quantite,
           e.client_id, e.date_commande, e.code_promo,
           l.quantite * l.prix_unitaire as montant_brut,
           l.quantite * l.prix_unitaire * (1 - l.taux_remise_ligne) as apres_remise_ligne
    from lignes l
    join entetes e using (commande_id)
```

```

),
-- 2) répartir la remise DE COMMANDE (Le code promo %) au prorata du brut de chaque ligne
avec_promo as (
    select b.*,
           coalesce(p.valeur, 0) / 100.0 as taux_promo, -- 10 → 0,10
           b.apres_remise_ligne * (1 - coalesce(p.valeur,0)/100.0) as montant_net
    from base b
    left join promos p on p.promo_code = b.code_promo and p.type_promo = 'PCT'
)
select commande_id, ligne_num, produit_id, client_id, code_promo, date_commande, quantite,
       montant_brut,
       montant_brut - montant_net as montant_remise,
       montant_net,
       montant_net / nullif(quantite,0) as prix_net_unitaire -- servira aux retours
from avec_promo

```

Vérifions à la main sur la commande 100 (promo 10 %) :

ligne	produit	qté	brut	net	prix_net_unitaire
100-1	Bouilloire	2	80,00	72,00	36,00
100-2	Tasse	4	48,00	43,20	10,80
101-1	Casque	1	100,00	95,00 (remise ligne 5 %)	95,00

Net de la commande 100 = 72,00 + 43,20 = 115,20 ✓ (= les paiements). Et le prix net unitaire de la bouilloire = 36,00, ce sera le montant remboursé du retour, pas 40,00.

int_paiements_commande — *le fan-in (raison 2)*. Une commande a plusieurs paiements ; on les réduit à une ligne par commande (somme des paiements OK) pour un contrôle de cohérence :

```

-- models/intermediate/int_paiements_commande.sql (ephemeral)
select commande_id,
       sum(case when statut = 'OK' then montant else 0 end) as montant_paye
from {{ ref('stg_paiement') }}
group by commande_id

```

→ commande 100 : 100,00 + 15,20 = 115,20 ; commande 101 : 95,00.

int_expeditions_detaillées — *le fan-in inter-tables*. La ligne d'expédition (exp_id, lig_no, qte_exp) ne connaît pas le produit : il est dans commande_ligne. On joint pour ramener le produit_id et l'entrepôt au grain de la ligne expédiée (le futur grain de fct_expeditions) :

```
-- models/intermediate/int_expeditions_detaillees.sql (ephemeral)
with el as ( select * from {{ ref('stg_expedition_ligne') }} ),
     e as ( select * from {{ ref('stg_expedition') }} ),
     cl as ( select commande_id, ligne_num, produit_id from {{ ref('stg_commande_ligne') }} )
select e.expedition_id, e.commande_id, e.entrepot_id, e.date_expedition,
       el.ligne_num, cl.produit_id, el.quantite_expediee,
       datediff('day', c.date_commande, e.date_expedition) as delai_traitement_jours
from el
join e using (expedition_id)
join cl on cl.commande_id = e.commande_id and cl.ligne_num = el.ligne_num
join {{ ref('stg_commande') }} c on c.commande_id = e.commande_id
```

Ces trois `int_*` ne créent aucun objet Snowflake (ils sont `ephemeral`). Ils existent uniquement pour factoriser le pipeline. Vous les verrez comme des CTE dans le SQL compilé des marts, jamais comme des tables. C’est exactement l’usage prévu au Cours 03.

4.B Les MARTS, découpés par domaine

On range maintenant les étoiles dans deux schémas de marts, selon le domaine (la bus matrix de l’Étape 2). Le `dbt_project.yml` mappe chaque dossier à son schéma, c’est là que « plusieurs marts » et « autres types de schémas » deviennent concrets :

```
# dbt_project.yml (extrait) – un dossier = un schéma
models:
  aureole:
    staging:
      +materialized: view
      +schema: staging
    staging_web:
      +materialized: view
      +schema: staging_web
    intermediate:
      +materialized: ephemeral
  marts:
    ventes:
      +materialized: table
      +schema: marts_ventes
    logistique:
      +materialized: table
      +schema: marts_logistique
    marketing:
      +materialized: table
      +schema: marts_marketing
  presentation:
```

```
+materialized: table
+schema: presentation
```

C'est le même moule qu'ailleurs (type de ressource, projet, dossier, réglages en + ; Théorie 03, section 2), mais poussé d'un cran : marts : ne porte aucun réglage, il ne fait que contenir trois sous-dossiers qui, eux, en portent. Rien n'empêcherait de poser +materialized: table sur marts : et de laisser les trois domaines n'indiquer que leur schéma ; c'est la cascade, et elle fonctionne sur autant de niveaux qu'on veut.

(La macro generate_schema_name du Cours 03 garantit les noms tels quels, sans préfixe DEV_.)

Schéma	Modèles	Domaine
MARTS_VENTES	dim_date, dim_client, dim_produit, dim_promotion, fct_ventes, fct_retours	ventes / finance
MARTS_LOGISTIQUE	fct_expeditions (+ dim_entrepot)	opérations
MARTS_MARKETING	fct_pages_vues (source web)	web / marketing

Les dimensions conformes (dim_date, dim_produit, dim_client) sont définies une seule fois dans MARTS_VENTES et référencées par la logistique et le marketing, pas de copie. Les clés de substitution sont par hachage (dbt_utils.generate_surrogate_key, Cours 04), donc chaque fait recalcule ses clés sans joindre les dimensions, quel que soit le schéma où il vit, et même s'il vient d'une autre source (le fait web hache produit_id et retombe sur la même clé que la dim_produit bâtie depuis l'appli).

fct_ventes lit l'intermediate (pas le staging brut) :

```
-- models/marts/ventes/fct_ventes.sql → schéma MARTS_VENTES
select
  to_number(to_char(date_commande, 'YYYYMMDD')) as date_key,
  {{ dbt_utils.generate_surrogate_key(['client_id']) }} as client_key,
  {{ dbt_utils.generate_surrogate_key(['produit_id']) }} as produit_key,
  {{ dbt_utils.generate_surrogate_key(['code_promo']) }} as promotion_key,
  quantite, montant_brut, montant_remise, montant_net
from {{ ref('int_lignes_nettes') }} -- ← La logique nette, réutilisée
```

fct_retours réutilise int_lignes_nettes pour retrouver le prix net unitaire, d'où le remboursement au bon montant :

```
-- models/marts/ventes/fct_retours.sql → schéma MARTS_VENTES
with retours as ( select * from {{ ref('stg_retour') }} ),
  nettes as ( select commande_id, ligne_num, produit_id, prix_net_unitaire
              from {{ ref('int_lignes_nettes') }} ) -- ← même source que les ventes
select
  to_number(to_char(r.date_retour, 'YYYYMMDD')) as date_key,
  {{ dbt_utils.generate_surrogate_key(['n.produit_id']) }} as produit_key, -- même hachage → dim conforme
```

```

    r.motif,
    r.quantite_retournee,
    r.quantite_retournee * n.prix_net_unitaire          as montant_rembourse
from retours r
join nettes n on n.commande_id = r.commande_id and n.ligne_num = r.ligne_num

```

→ retour 700 : $1 \times 36,00 = 36,00$ \$. La cohérence tient parce que ventes et retours partagent int_lignes_nettes : impossible qu'ils calculent le net différemment.

fct_expeditions vit dans l'autre mart et lit son intermediate :

```

-- models/marts/logistique/fct_expeditions.sql → schéma MARTS_LOGISTIQUE
select
    to_number(to_char(date_expedition, 'YYYYMMDD'))          as date_key,
    {{ dbt_utils.generate_surrogate_key(['produit_id']) }} as produit_key,      -- dim conforme
    {{ dbt_utils.generate_surrogate_key(['entrepot_id']) }} as entrepot_key,
    quantite_expediee,
    delai_traitement_jours
from {{ ref('int_expeditions_detaillees') }}

```

fct_pages_vues vit dans le 3^e mart et lit le staging de l'autre source, sans aucun int_ (rien à assembler ici) :

```

-- models/marts/marketing/fct_pages_vues.sql → schéma MARTS_MARKETING
select
    to_number(to_char(date_vue, 'YYYYMMDD'))          as date_key,
    {{ dbt_utils.generate_surrogate_key(['produit_id']) }} as produit_key,      -- MÊME hachage → dim conforme, AUTRE source
    session_id,
    duree_sec
from {{ ref('stg_web_page_vue') }}

```

Le point à voir : produit_key est haché depuis produit_id, qui vient ici du web (le SKU). Comme dim_produit a haché le même produit_id (venu de l'appli), les deux coïncident : le fait web se relie tout seul à une dimension bâtie depuis une autre source. C'est la dimension conforme inter-sources, en une ligne, le cœur de ce que cette synthèse ajoute.

Les faits, une fois bâtis (on retrouve nos chiffres) :

MARTS_VENTES.FCT_VENTES

date_key	client_key	produit_key	quantite	montant_brut	montant_net
20240510	(Alice)	(Bouilloire)	2	80,00	72,00
20240510	(Alice)	(Tasse)	4	48,00	43,20

date_key	client_key	produit_key	quantite	montant_brut	montant_net
20240512	(Bruno)	(Casque)	1	100,00	95,00

MARTS_LOGISTIQUE.FCT_EXPEDITIONS (la commande 100 sur 2 entrepôts, la tasse en 2 fois)

date_key	produit_key	entrepot_key	quantite_expediee	delai_traitement_jours
20240511	(Bouilloire)	(Montréal)	2	1
20240511	(Tasse)	(Montréal)	2	1
20240513	(Tasse)	(Toronto)	2	3
20240513	(Casque)	(Toronto)	1	1

MARTS_VENTES.FCT_RETOURS

date_key	produit_key	motif	quantite_retournee	montant_rembourse
20240520	(Bouilloire)	DEF	1	36,00

MARTS_MARKETING.FCT_PAGES_VUES (source web ; produit_key conforme à celui des ventes)

date_key	produit_key	session_id	duree_sec
20240509	(Bouilloire)	S-1	42
20240510	(Casque)	S-2	88
20240510	(Bouilloire)	S-2	15

La Bouilloire porte le même produit_key ici que dans FCT_VENTES et FCT_RETOURS : on peut donc dire « consultée 2 fois, vendue 2, retournée 1 » en croisant les trois faits sans rien réconcilier, deux sources, une seule vérité produit.

4.C Les trois piliers : SCD2, tests, doc

Snapshot (SCD2) — on historise dim_client sur le segment et la province, stratégie check (pas d’horodatage source), comme Boréal/La Brûlerie :

```
-- snapshots/scd_client.sql → schéma SNAPSHOTS
{% snapshot scd_client %}
{{ config(target_schema='snapshots', unique_key='client_id',
          strategy='check', check_cols=['segment','province']) }}
select client_id, segment, province from {{ ref('stg_client') }}
{% endsnapshot %}
```

Tests — la nouveauté, c'est que relationships traverse deux schémas de marts : le fait de la logistique doit pointer vers la dim_produit conforme, définie dans les ventes. Le test le prouve :

```
# models/marts/logistique/_Logistique.yml
models:
  - name: fct_expeditions
    columns:
      - name: produit_key
    tests:
      - not_null
      - relationships:                # Le gardien, à travers Les domaines
        arguments:
          to: ref('dim_produit')      # ← dim conforme, dans MARTS_VENTES
          field: produit_key
```

Le même test garde la conformité inter-sources : fct_pages_vues.produit_key (source web) doit exister dans dim_produit (bâtie depuis l'appli), un relationships qui, s'il passe, prouve que les deux systèmes parlent bien du même produit.

Et un test singulier qui exploite l'intermediate int_paiements_commande, les ventes nettes d'une commande doivent éгалer ce qui a été payé :

```
-- tests/assert_ventes_egalent_paiements.sql (0 ligne = réussi)
with v as ( select commande_id, sum(montant_net) as net
            from {{ ref('int_lignes_nettes') }} group by commande_id ),
     p as ( select * from {{ ref('int_paiements_commande') }} )
select v.commande_id, v.net, p.montant_paye
from v join p using (commande_id)
where abs(v.net - p.montant_paye) > 0.01
```

→ commande 100 : 115,20 = 115,20 ✓ ; commande 101 : 95,00 = 95,00 ✓. Zéro ligne : la donnée est réconciliée.

Documentation et exposure — chaque modèle a sa description ; dbt docs generate produit le lignage cliquable, on y voit les int_* en CTE et les deux marts se brancher sur les mêmes dimensions conformes. Un exposure déclare les futurs rapports Power BI (Cours 06).

Tout lancer — une commande orchestre l'ensemble, dans l'ordre du DAG :

```
dbt build      # run + test + snapshot, staging → intermediate → marts (Les 2 schémas)
```

Avant-goût — la couche PRESENTATION (*approfondie au Cours 06*)

Au-dessus des marts, on ajoute parfois une couche PRESENTATION (ou *reporting*) : des tables pré-agrégées, taillées pour un rapport précis, que Power BI lit sans recalculer. Ce n'est pas l'étoile (on quitte le grain fin), c'est un résumé de service. Un exemple, les ventes nettes mensuelles par catégorie :

```
-- models/presentation/rpt_ventes_mensuelles.sql → schéma PRESENTATION
select d.annee, d.mois, p.categorie,
       sum(f.montant_net) as ventes_nettes,
       sum(f.quantite)   as unites_vendues
from {{ ref('fct_ventes') }} f
join {{ ref('dim_date') }}   d using (date_key)
join {{ ref('dim_produit') }} p using (produit_key)
group by d.annee, d.mois, p.categorie
```

C'est un autre type de schéma encore, la démonstration finale que RAW/STAGING/MARTS n'épuise pas la liste. On s'arrête ici : brancher, modéliser et publier ce genre de table côté Power BI, c'est le Cours 06.

Récapitulatif — ce que la synthèse a rejoué, et ce qu'elle a montré en plus

Démarche (cours)	Où, sur Aurore
Cadrer le projet + l'architecture en couches (00)	Étape 0 (3 processus, plan des schémas)
Charger le brut dans RAW (01)	Étape 1 (13 tables OLTP appli + flux web)
Plusieurs sources → plusieurs RAW/STAGING	Étapes 1.2 + 3 (RAW_WEB/STAGING_WEB, un schéma par source)
Concevoir l'étoile — 4 décisions de Kimball (02)	Étape 2, ×4 processus → constellation
Grain le plus fin, additivité, SCD (02)	Étape 2 (grains ; délai non additif ; SCD2 client)
Dimensions conformes + bus matrix (02)	Étape 2.2 (la grille pilote le découpage)
Dimension conforme <i>entre sources</i>	Étape 4.B (fct_pages_vues web ↔ dim_produit appli, même hachage)
STAGING — vues 1:1, renommer/typer/décoder (03)	Étape 3 (13 stg_*)
INTERMEDIATE — int_*, ephemeral (03, mentionné)	Étape 4.A (réutilisation + fan-in)
MARTS en table, clés par hachage (04)	Étape 4.B
Plusieurs marts par domaine (mappés dans dbt_project.yml)	Étape 4.B (MARTS_VENTES / MARTS_LOGISTIQUE / MARTS_MARKETING)
Snapshot SCD2 (04)	Étape 4.C (scd_client)
Tests relationships / singuliers (04)	Étape 4.C (relationship inter-schémas ; réconciliation)

Démarche (cours)	Où, sur Aurore
Doc, lignage, exposure, dbt build (04)	Étape 4.C
Couche PRESENTATION (autre type de schéma)	Avant-goût (rpt_*)

Les « types de schémas » rencontrés : deux RAW et deux STAGING (un par source : appli + web) · INTERMEDIATE (réutilisable, éphémère) · trois MARTS par domaine, reliés par des dimensions conformes (y compris entre sources) · PRESENTATION (agrégats de rapport) · SNAPSHOTS (historique). La colonne vertébrale RAW → STAGING → MARTS tient toujours, Aurore lui a seulement ajouté les couches et les dédoublements (par source et par domaine) que sa complexité réclamait. C'est la leçon à emporter : on structure autant que le projet l'exige, pas plus.

Le corrigé complet est à côté, dans Solution/ : `Projet_Aurore.sql` pour le SQL Snowflake (les deux schémas de brut, les 14 tables, les `COPY INTO`) et `Solution/dbt/` pour le projet dbt entier, les 14 modèles de staging, les trois `int_*`, les trois marts et la couche de présentation. Le `README.md` du dossier signale ce qui y a été écrit et que la synthèse n'expose pas.